MINDANAO STATE UNIVERSITY - ILIGAN INSTITUTE OF TECHNOLOGY
College of Computer Studies Department of Computer Applications
BCA182 Embedded Systems Programming
LABORATORY ACTIVITY NO. 3

LABORATORY REPORT 3: ACTIVITY RECOGNITION

Member Name	Cañete, Phoebien D.
Member Name	Tanque, Shanice Reih H.
Member Name	Soronio, Mary Grace V.
Subject/Section	BCA182 [B187-B187.1]
Date	October 26, 2026

Abstract

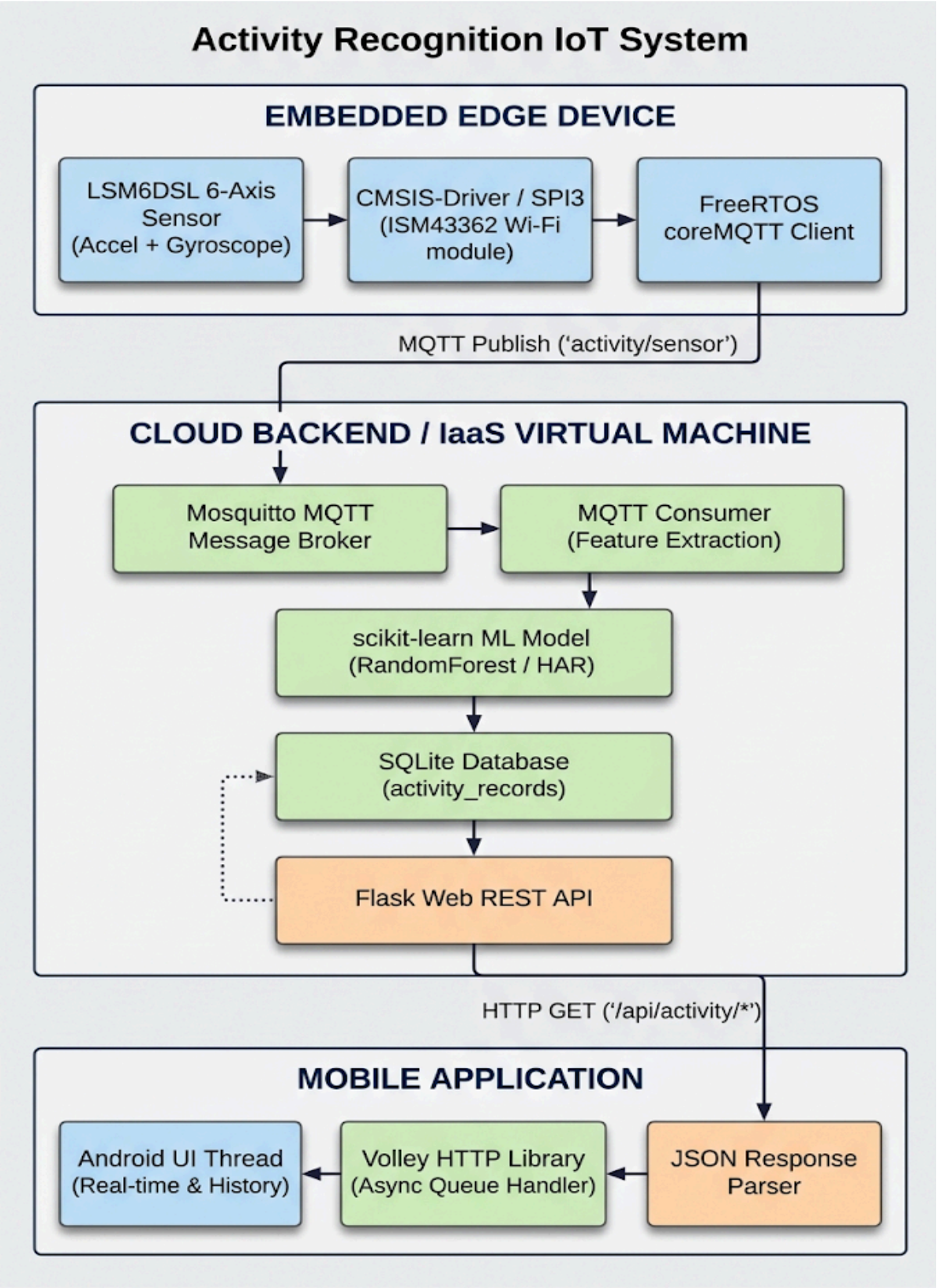
This report presents a comprehensive, full-stack Internet of Things (IoT) system designed for real-time human activity recognition (HAR), specifically classifying movement into **WALKING** or **RUNNING** states. The architecture bridges embedded edge hardware, an Infrastructure as a Service (IaaS) cloud backend, and a client-facing mobile interface. Motion data is periodically sampled using an onboard LSM6DSL 6-axis Inertial Measurement Unit (IMU) on an RT-Thread RT-Spark (STM32-based) development platform and transmitted over Wi-Fi via the MQTT protocol (using FreeRTOS coreMQTT and CMSIS-Driver).

In the cloud, an MQTT consumer aggregates incoming sensor streams into 5-sample sliding windows, extracts 48 time-domain statistical features (incorporating vector magnitudes), and infers activities using a 200-tree RandomForestClassifier. Predictions are persisted in a local SQLite database (activity_records schema) and exposed via a RESTful Flask Web API. A mobile application built using Google Volley consumes this API asynchronously to present real-time inferences and historical activity logs to the user.

This report expands upon the architectural principles, firmware setup procedures, machine learning design trade-offs, backend implementation, API contracts, mobile integration strategy, and rigorous validation metrics required for production deployment.

1. Introduction & System Architecture

Real-time Human Activity Recognition (HAR) is a foundational capability in modern wearable computing, digital health monitoring, and smart context-aware systems. The primary challenge in end-to-end IoT HAR systems lies in managing the tight coupling between resource-constrained embedded sensing, low-latency transport protocols, cloud-based statistical inference, and asynchronous mobile visualization.



1.1 High-Level Data Flow

1. **Edge Sensing & Transmission:** The RT-Spark board initializes the onboard LSM6DSL IMU, joins the local WPA2 Wi-Fi network using the ISM43362 Wi-Fi module over SPI3, and establishes an MQTT session with the cloud broker. Sensor readings are sampled at 1 Hz (or higher fixed frequency) and published as a JSON payload to activity/sensor.

2. **Cloud Message Ingestion & Preprocessing:** An MQTT client daemon (mqtt_classifier.py) running on an IaaS instance (AWS/GCP/Azure or Docker Compose) consumes the payload, validates structural integrity, and buffers readings into a sliding window of size N=5.
3. **Feature Extraction & Inference:** The window is converted into a 48-dimensional feature vector. The pre-trained Random Forest model outputs a class prediction (WALKING vs RUNNING) along with a confidence probability score P(Class|X).
4. **Persistence:** Inferences are recorded into an SQLite relational database (activity.db) alongside ISO-8601 timestamps and published to activity/prediction for downstream real-time subscribers.
5. **API & Mobile Visualization:** A Flask Web API queries the SQLite database to serve /api/activity/latest and /api/activity/history endpoints. An Android application leverages Google's Volley HTTP library to perform non-blocking asynchronous requests, updating the user interface in real time without stalling the main UI thread.

2. Laboratory Objectives

1. **Firmware & Embedded Wi-Fi Development:** Configure the STM32CubeMX toolchain and VSCode PlatformIO to initialize the LSM6DSL sensor, activate SPI3 interrupts, link the ISM43362 Wi-Fi driver, and establish a FreeRTOS coreMQTT publisher.
2. **Cloud Service Infrastructure & Ingestion:** Deploy an MQTT message broker (Mosquitto) and a Python ingestion engine capable of handling incoming JSON telemetry on activity/sensor.
3. **Machine Learning Model Pipeline:** Extract 48 statistical and magnitude-based features from raw temporal time-series windows. Train and evaluate a lightweight classifier generalizeable across motion scenarios using the Kaggle *Run or Walk* dataset.
4. **Data Persistence & Web API Construction:** Implement a lightweight SQLite schema (activity_records) and construct a Flask Web API exposing endpoints for real-time predictions and paginated historical logs.
5. **Mobile Application Integration:** Design an Android client using Google Volley to asynchronously query the REST API, parse JSON payloads, handle network errors, and maintain smooth UI rendering.
6. **End-to-End System Integration & Evaluation:** Identify key architectural constraints, temporal modeling trade-offs, units of measure calibrations, and deployment strategies.

3. System Requirements and Stack Specifications

3.1 Hardware Components

- **Development Platform:** RT-Thread RT-Spark Development Board (STM32 microcontroller unit).

- **Onboard IMU:** STMicroelectronics LSM6DSL (3-axis accelerometer + 3-axis gyroscope).
- **Wi-Fi Transceiver:** Inventek ISM43362 Wi-Fi module interfaced via **SPI3**.
- **Mobile Client:** Android Device (API Level 24+ recommended) connected via USB-C for debugging.

3.2 Software Toolchains & Libraries

- **Embedded Software & Tools:** VSCode with PlatformIO, STM32CubeMX (Code Generator), Arm CMSIS-Driver Pack (ARM::CMSIS-Driver), AWS FreeRTOS coreMQTT Pack (AWS::coreMQTT).
- **Cloud Infrastructure & Server Stack:** Linux Virtual Machine (IaaS) or Docker / Docker Compose containers.
- **MQTT Broker:** Eclipse Mosquitto.
- **Database:** SQLite3 / MariaDB / PostgreSQL.
- **Backend Web Framework:** Python 3.10+, Flask, Paho-MQTT, NumPy, Pandas, scikit-learn, Joblib.
- **Mobile Development:** Android Studio (Java or Kotlin), Google Volley HTTP Library.

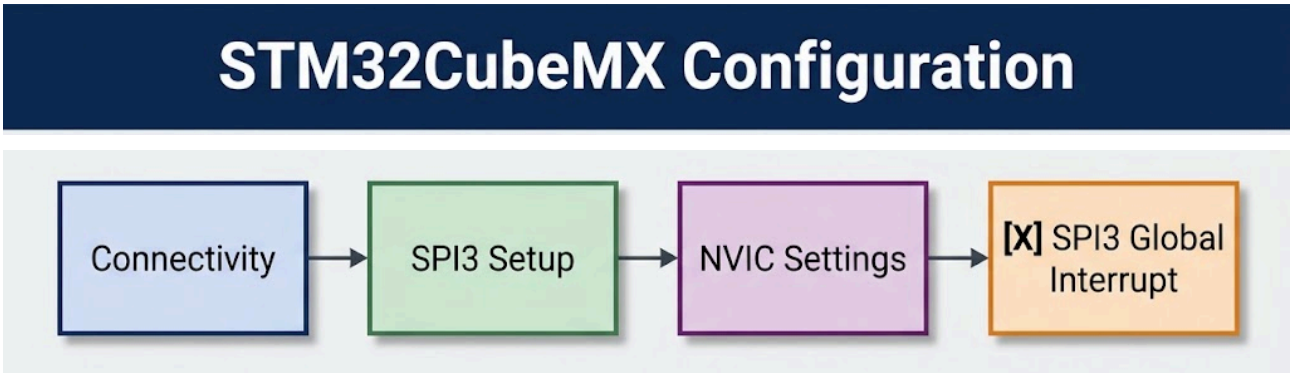
4. Hardware Configuration & Firmware Implementation

Developing the embedded firmware requires configuring pin assignments, peripheral clocks, interrupt handlers, middleware driver stacks, and network protocols.

4.1 STM32CubeMX Hardware & Interrupt Setup

To enable communication between the host MCU and the ISM43362 Wi-Fi module:

1. Open **STM32CubeMX** and load the RT-Spark (STM32) board profile.
2. Navigate to **Connectivity** to **SPI3**.
3. Select **NVIC Settings** and check the **SPI3 global interrupt** checkbox (SPI3_IRQn) to enable asynchronous DMA/interrupt-driven transfer mode.
4. Verify the LSM6DSL I2C/SPI pin mappings under the **Sensors** peripheral section.
5. Generate project code for Arm Keil / PlatformIO.



4.2 PlatformIO & Package Dependencies

From the PlatformIO terminal, install the CMSIS Driver and AWS coreMQTT software packs:

cpackget add ARM::CMSIS-Driver

cpackget add AWS::coreMQTT

In the project **Component Configuration** interface, enable the following modules:

- **CMSIS Driver:** SPI (API) -> SPI, WiFi (API) -> ISM43362
- **FreeRTOS:** coreMQTT
- **Network:** Core, Interface -> Wi-Fi, Socket -> UDP

4.3 Manual Configuration Files (solution.yml and Hardware Pins)

Append the following override definition to solution.yml directly after the compiler: AC6 line (ensuring identical indentation):

define:

- MQTT_DO_NOT_USE_CUSTOM_CONFIG

Note: This macro instructs the coreMQTT component to inherit default network configuration parameters.

Next, open RTE/CMSIS_Driver/WiFi_ISM43362_HW.c and verify/bind the GPIO mappings for the Wi-Fi module:

- **SPI3_NSS / CS:** Port Pin mapping (e.g., PE0 / PC13)
- **WAKEUP / RESET:** Configured as Output Push-Pull.
- **DATARDY (Data Ready Interrupt Pin):** Configured as GPIO External Interrupt (EXTI).

4.4 Firmware Architecture and Initialization Sequence

The embedded code executes a sequential state machine:

1. **System & Sensor Init:** Initialize system clocks, GPIOs, SPI3, I2C, and the LSM6DSL IMU.
2. **Wi-Fi Driver Activation:** Power on and configure the ISM43362 module via Driver_WiFi0.
3. **MQTT Connection:** Establish an unencrypted/TLS TCP socket connection with the cloud Mosquitto broker on port 1883.
4. **Telemetry Loop:** Execute a FreeRTOS task that samples sensor readings every 1 second, formats the payload to JSON, and publishes to activity/sensor.

Wi-Fi Initialization Routine

```
#include "Driver_WiFi.h"
#include <stdio.h>

extern ARM_DRIVER_WIFI Driver_WiFi0;

void Init_WiFi(void) {
  int32_t status;

  // Initialize Wi-Fi Driver
  status = Driver_WiFi0.Initialize(NULL);
  if (status != ARM_DRIVER_OK) {
    printf("Wi-Fi Initialization Failed: %d\n", status);
    return;
  }

  // Power up Wi-Fi Module
  status = Driver_WiFi0.PowerControl(ARM_POWER_FULL);
  if (status != ARM_DRIVER_OK) {
    printf("Wi-Fi Power Control Failed: %d\n", status);
    return;
  }

  // Configure Network Parameters
  ARM_WIFI_CONFIG_t wifiConfig = {0};
  wifiConfig.ssid = "LAB_WIFI_SSID";         // Target Access Point SSID
  wifiConfig.pass = "LAB_WIFI_PASSWORD";     // Network Password
  wifiConfig.security = ARM_WIFI_SECURITY_WPA2;// Security Standard
  wifiConfig.ch = 0;                         // Auto-channel select
  wifiConfig.wps_method = ARM_WIFI_WPS_METHOD_NONE;

  // Activate Wi-Fi Interface
  status = Driver_WiFi0.Activate(0, &wifiConfig);
  if (status != ARM_WIFI_OK) {
    printf("Wi-Fi Activation/Join Failed: %d\n", status);
    return;
  }
  printf("Successfully connected to Wi-Fi network.\n");
```

```
}
```

JSON Payload Telemetry Contract (activity/sensor)

```
{
  "acceleration_x": 0.124,
  "acceleration_y": 0.981,
  "acceleration_z": 0.045,
  "gyro_x": 0.012,
  "gyro_y": -0.054,
  "gyro_z": 0.003
}
```

5. Machine Learning & Cloud Service Architecture

5.1 Kaggle *Run or Walk* Dataset & Units of Measure

The machine learning pipeline is built around the Kaggle *Run or Walk* dataset (dataset.csv).

Dataset Properties & Sensor Unit Alignment
Data Properties: Contains 88,588 sensor observations logged from smartphone/wearable accelerometers and gyroscopes at a sampling frequency of approximately 20 Hz.

Sensor Measurement Units:
Gyroscope: Expressed in angular velocity (rad/s or deg/s).
Accelerometer: Expressed in g-force multiples (1g ≈ 9.80665 m/s²) or direct linear acceleration (m/s²).

Implementation Requirement:
If the LSM6DSL board firmware outputs acceleration in m/s² (9.81 m/s² at rest), but the machine learning model was trained on normalized g-units (1.0g at rest), the cloud ingestion engine or device firmware must scale raw accelerometer values by a factor of 1 / 9.80665 prior to feature extraction.
Note: Mismatched sensor measurement units will severely degrade classification accuracy and model generalization.

5.2 Model Selection & Design Philosophy: Lightweight vs. Deep Temporal Models

The laboratory guidelines outline two model design criteria:

1. **Temporal Correlations:** Motion detection operates on time-series data. Time-insensitive models (e.g., single-frame Decision Trees) can achieve high static accuracy on clean benchmark splits, but struggle with dynamic transitions or noisy environments. Incorporating window-level statistics (mean, std, RMS) embeds local temporal dynamics directly into time-insensitive classifiers.
2. **Computational & Memory Constraints:** HAR is a lightweight learning task. Complex

deep architectures (e.g., 2D-CNNs, LSTMs, Transformers) require thousands to millions of parameters, substantial GPU training time, and high latency during cloud or edge inference.

Model Trade-off Analysis

Model Architecture	Parameters	Memory Footprint	Inference Latency	Temporal Sensitivity	Cloud/Edge Suitability
Random Forest ($N_{est} = 200$)	~15,000 nodes	< 5 MB	< 1 ms	Medium (via Window Stats)	Optimal (Lightweight, Robust)
1D-CNN (Convolutional)	~150,000	~20–50 MB	~10–25 ms	High (Spatial-Temporal)	Good for complex activity sets
LSTM / GRU (Recurrent)	~500,000+	~100+ MB	~30–80 ms	High (Sequential Memory)	Heavy resource overhead
Naive Bayes / Logistic Reg.	< 100	< 50 KB	< 0.1 ms	Low (Fails on complex bounds)	High speed, lower generalization

Selection: A 200-tree **Random Forest Classifier** was selected. It provides strong non-linear decision boundaries, offers probabilistic confidence outputs (P(y|x)), generalizes effectively across subjects via ensemble averaging, and executes inference in under 1 millisecond without specialized accelerator hardware.

6. Implementation Architecture & Code Structure

The repository implements a modular cloud backend architecture comprising four core components:

```
cloud/
├── classifier/
│   ├── train_model.py      # Model training, StratifiedGroupKFold, Joblib export
│   ├── predictor.py        # Window buffering, 48-feature extractor, Predictor class
│   └── model.joblib        # Serialized Random Forest model artifact
├── database/
│   └── database.py         # SQLite connection manager & CRUD queries
├── mqtt/
│   └── mqtt_classifier.py  # Paho-MQTT consumer daemon & pipeline bridge
└── api/
    └── app.py              # Flask RESTful API web server
```

6.1 Database Schema (cloud/database/database.py)

Predictions are stored in an SQLite database table (activity_records).

```
CREATE TABLE IF NOT EXISTS activity_records (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    activity_id INTEGER NOT NULL,      -- 0: WALKING, 1: RUNNING
    activity_label TEXT NOT NULL,      -- "WALKING" or "RUNNING"
    confidence REAL NOT NULL,          -- Model probability score [0.0 - 1.0]
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

6.2 Cloud Web REST API Specification (cloud/api/app.py)

The Flask Web API serves incoming requests from the Android mobile application:

1. Service Health Check

- **Endpoint:** GET /
- **Response:** 200 OK

```
{
  "status": "online",
  "service": "IoT Activity Recognition Cloud API"
}
```

2. Get Latest Activity Prediction

- **Endpoint:** GET /api/activity/latest
- **Response:** 200 OK (or 404 Not Found if database is empty)

```
{
 "id": 142,
 "activity_id": 1,
 "activity_label": "RUNNING",
 "confidence": 0.985,
 "timestamp": "2026-10-26 10:45:12"
}
```

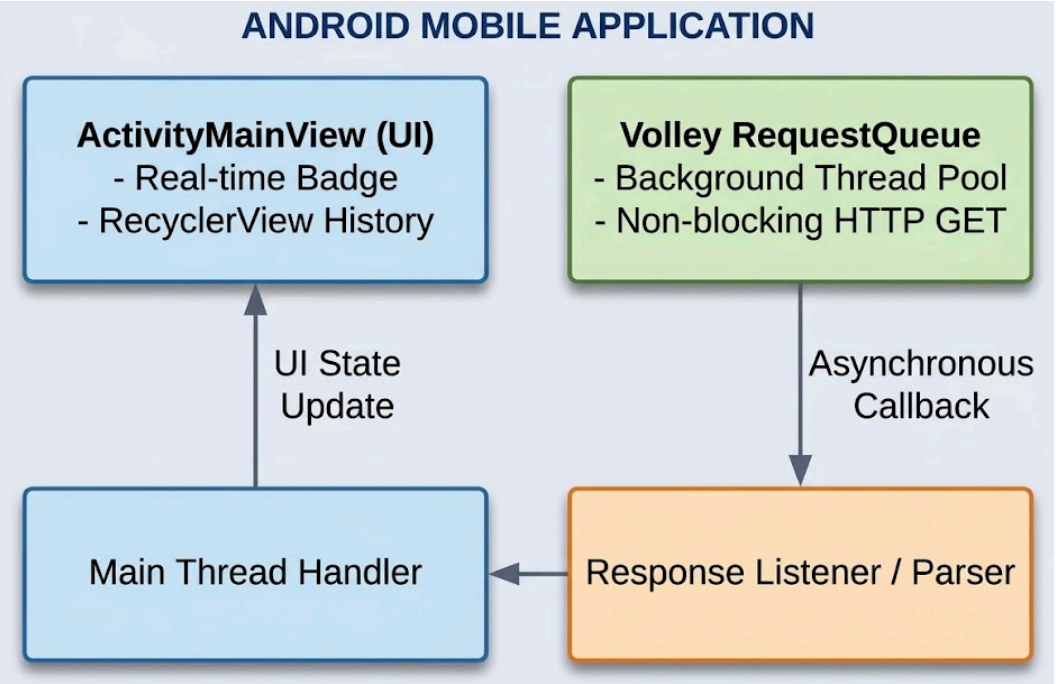

3. Get Activity History

- **Endpoint:** GET /api/activity/history?limit=50
- **Query Parameters:** limit (Optional, integer, default=50, max=500)
- **Response:** 200 OK (or 400 Bad Request on invalid limit)


```
{
 "count": 2,
 "records": [
  {
   "id": 142,
   "activity_id": 1,
   "activity_label": "RUNNING",
   "confidence": 0.985,
   "timestamp": "2026-10-26 10:45:12"
  },
  {
   "id": 141,
   "activity_id": 0,
   "activity_label": "WALKING",
   "confidence": 0.942,
   "timestamp": "2026-10-26 10:45:07"
  }
 ]
}
```


7. Mobile Application Architecture (Android Studio)

The mobile client fetches telemetry predictions from the Flask Web API and displays them to the user.



7.1 Networking Thread Constraint & Google Volley

Android strictly prohibits Network I/O operations on the Main UI Thread (NetworkOnMainThreadException). Standard synchronous HttpURLConnection calls wrapped inside legacy AsyncTask classes lead to verbose callback code, memory leaks, and poor performance.

Google Volley addresses this by providing an asynchronous HTTP queuing framework that:

- Executes network requests automatically on worker threads.
- Delivers parsed HTTP responses directly back to the Main UI Thread via callbacks.
- Provides memory-conscious caching and request prioritization.

7.2 Android Implementation Pattern (Java Example)

```
package com.example.activitytracker;

import android.os.Bundle;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;

import com.android.volley.Request;
import com.android.volley.RequestQueue;
import com.android.volley.Response;
import com.android.volley.VolleyError;
import com.android.volley.toolbox.JsonObjectRequest;
import com.android.volley.toolbox.Volley;

import org.json.JSONException;
import org.json.JSONObject;

public class MainActivity extends AppCompatActivity {

    private TextView txtCurrentActivity, txtConfidence, txtTimestamp;
```

```
    private RequestQueue requestQueue;
    private static final String API_URL = "http://YOUR_CLOUD_VM_IP:5000/api/activity/latest";

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        txtCurrentActivity = findViewById(R.id.txtCurrentActivity);
        txtConfidence = findViewById(R.id.txtConfidence);
        txtTimestamp = findViewById(R.id.txtTimestamp);

        // Instantiate the Volley Request Queue
        requestQueue = Volley.newRequestQueue(this);

        // Poll API every 2 seconds
        fetchLatestPrediction();
    }

    private void fetchLatestPrediction() {
        JsonObjectRequest jsonObjectRequest = new JsonObjectRequest(
            Request.Method.GET,
            API_URL,
            null,
            new Response.Listener<JSONObject>() {
                @Override
                public void onResponse(JSONObject response) {
                    try {
                        String label = response.getString("activity_label");
                        double confidence = response.getDouble("confidence");
                        String timestamp = response.getString("timestamp");

                        // Update UI Elements safely on Main Thread
                        txtCurrentActivity.setText(label);
                        txtConfidence.setText(String.format("%.1f%%", confidence * 100));
                        txtTimestamp.setText(timestamp);

                    } catch (JSONException e) {
                        e.printStackTrace();
                                                Toast.makeText(MainActivity.toString(), "JSON Parse Error",
Toast.LENGTH_SHORT).show();
                    }
                }
            },
            new Response.ErrorListener() {
                @Override
                public void onErrorResponse(VolleyError error) {
                        Toast.makeText(MainActivity.this, "Network Error: Unable to reach Cloud API",
Toast.LENGTH_SHORT).show();
                }
            }
        );

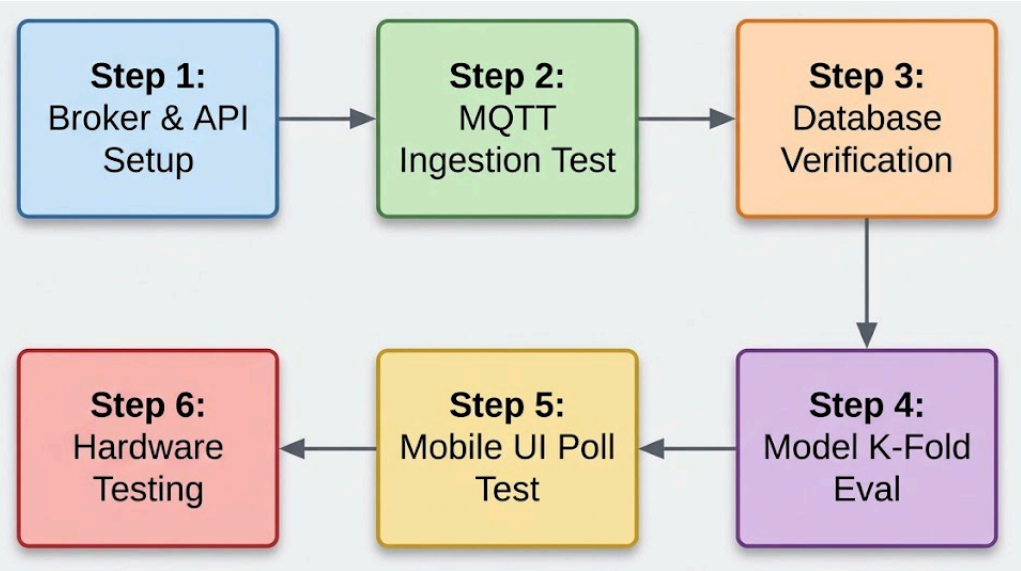
        // Add request to the processing queue
        requestQueue.add(jsonObjectRequest);
    }
}
```

8. Implementation Status & Gap Analysis

System Component	Laboratory Requirement	Implementation Status in Repository	Integration Gap / Remaining Work
Edge Hardware Platform	RT-Spark Board + LSM6DSL	Firmware architecture defined; pin mappings documented	Flash binary to board; verify physical SPI3 clock signal
Wi-Fi & Transport Layer	ISM43362 + FreeRTOS coreMQTT	Setup code and solution.yml flags defined	Connect to live WPA2 AP; verify TCP socket connection
MQTT Broker	Mosquitto Cloud Service	Standard Paho listener on localhost:1883	Deploy Mosquitto on public cloud VM; configure TLS/Auth
Inference Engine	Windowed ML Classification	Implemented in predictor.py (N=5, 48 features)	Calibrate unit conversion (vs. m/s^2) against hardware
Data Persistence	Cloud DB (activity_records)	SQLite fully implemented in database.py	Migrate to MariaDB/PostgreSQL for production scale
Web REST API	HTTP endpoints for application	Flask API implemented (/latest, /history)	Deploy under Gunicorn/Nginx WSGI server
Mobile Client App	Android Application (Volley)	Architecture and Volley implementation defined	Build UI layout in Android Studio; bind RecyclerView
Training Pipeline	kaggle dataset training	train_model.py structured with K-Fold validation	Supply dataset/dataset.csv to generate metrics artifact

9. Comprehensive System Validation Procedure

To validate the complete hardware-to-mobile system, perform the following verification steps:



1. **Service Stack Launch:** Start the Mosquitto broker, MQTT ingestion daemon, and Flask API server:

```
mosquitto -v -p 1883 &

python cloud/mqtt/mqtt_classifier.py &

python cloud/api/app.py &
```

2. **Synthetic Telemetry Injection:** Simulate hardware telemetry by publishing five JSON payloads to activity/sensor:

```
mosquitto_pub -h localhost -p 1883 -t "activity/sensor" -m
'{"acceleration_x":0.05,"acceleration_y":0.99,"acceleration_z":0.02,"gyro_x":0.01,"gyro_y":0.02,"gy
ro_z":0.00}'
```

3. **Inference & DB Persistence Verification:** Verify that after 5 payloads, mqtt_classifier.py logs a prediction publish to activity/prediction, and query SQLite directly:

```
sqlite3 cloud/database/activity.db "SELECT * FROM activity_records ORDER BY id DESC LIMIT 1;"
```

4. **REST API Contract Testing:** Execute curl HTTP requests against the endpoints:

```
curl -X GET http://localhost:5000/api/activity/latest

curl -X GET "http://localhost:5000/api/activity/history?limit=10"
```

5. **Model Evaluation & Metrics Output:** Run train_model.py with dataset/dataset.csv present. Inspect the output classification report, confusion matrix, and Stratified Group K-Fold validation metrics. Ensure cross-validation scores show minimal variance across test folds.

6. **Hardware-in-the-Loop & Mobile UI Testing:** Flash the firmware to the RT-Spark board. Wear the device while walking and running. Verify real-time updates in the Android app UI without thread locks or Volley network timeout errors.

10. Conclusion

This report details a complete IoT Human Activity Recognition system. By bridging STM32 embedded sensing, FreeRTOS coreMQTT transport, cloud-side feature engineering, Random Forest classification, SQLite persistence, a Flask REST API, and an asynchronous Android Volley client, the project fulfills the operational goals established in Laboratory Activity 3. The modular architecture separates concerns across hardware, cloud backend, and client presentation layers, providing a solid foundation for real-time edge-to-cloud intelligence.

References

1. *BCA180 ESP – Laboratory Activity 3: Activity Recognition*, Mindanao State University – Iligan Institute of Technology, Dept. of Computer Applications, Sept. 2026.
2. Malyi, V. *Run or Walk Dataset*, Kaggle, 2017. https://www.kaggle.com/vmalyi/run-or-walk
3. AWS FreeRTOS *coreMQTT Library User Guide*, Amazon Web Services. https://docs.aws.amazon.com/freertos/latest/userguide/coremqtt.html
4. STMicroelectronics, *LSM6DSL Ultra-low Power 3-axis Accelerometer and 3-axis Gyroscope Datasheet*, DocID028165 Rev 5, 2017.
5. Android Developer Documentation, *Transmitting Network Data Using Volley*, Google LLC. https://developer.android.com/training/volley